Department of Computer Science
University of Pretoria

Programming Languages
COS 333

Practical 5: Fortran

22 April, 2026

1 Objectives

This practical aims to achieve the following general learning objectives:

- To gain and consolidate some experience writing programs in the imperative programming language Fortran;
- To consolidate a variety of basic concepts related to imperative programming languages, as presented in the prescribed textbook for this course.

2 Plagiarism Policy

Plagiarism is a serious form of academic misconduct. It involves both appropriating someone else's work and passing it off as one's own work afterwards. Thus, you commit plagiarism when you present someone else's written or creative work (words, images, ideas, opinions, discoveries, artwork, music, recordings, computer-generated work, etc.) as your own. Note that using material produced in whole or part by an AI-based tool (such as ChatGPT) also constitutes plagiarism. Only hand in your own original work. Indicate precisely and accurately when you have used information provided by someone else. Referencing must be done in accordance with a recognised system. Indicate whether you have downloaded information from the Internet. For more details, visit the library's website: `http://www.library.up.ac.za/plagiarism/`.

3 Submission Instructions

Upload your practical-related source code files to the appropriate assignment upload slots on the ClickUP course page. Your Fortran submission must be implemented and submitted in a single file named `s99999999.f`, where `99999999` is your student number. Multiple uploads are allowed, but only the last one will be marked. The submission deadline is **Monday, 11 May 2026, at 12:00**.

4 Background Information

For this practical, you will be writing programs in Fortran 2008. You will have to investigate this language in terms of its support for different concepts related to data types, control structures and subprograms. To do this, you will have to write a short program to demonstrate how the language handles the concepts under consideration. This program should not be long, but will demonstrate the concepts adequately, and should allow you to understand the language's support (or lack of support) for the features in question.

Your Fortran 2008 program should compile using the `gfortran` compiler. Support for Fortran 2008 is provided under Linux as part of the GNU Compiler Collection with the `gcc` compiler. If you decide to use a different compiler, **make sure that you test your program using the specified compiler**. The course ClickUP page contains documentation related to the Fortran language [1] and the `gfortran` compiler [2].

Note that that your Fortran program code must comply with the `gfortran` compiler's fixed format. Refer to the documentation for the compiler for information on how to enforce fixed format source files. Note that the specified file extension of your source code file (the `.f` extension, as specified in Section 3) affects whether fixed format is used by default. Adherence to the fixed format will contribute to your mark for your submission.

5 Practical Tasks

You have to implement a simple statistical measure program in Fortran 2008. The program reads in and stores a sequence of floating point values and then prints out some statistics related to these values. Note that in the following discussion function signatures are provided in C-like syntax, and are intended to illustrate only the basic nature of the return and parameter types. Your programs must adhere to the following requirements:

- You do not need to define any user-specified data types. Simply use an array to store your data.
- You must define a subprogram with the signature `void readData(arr)`, where `arr` is an array of floating point values. The `readData` subprogram should prompt the user for floating point inputs and populate the array with these values. The array should be modified by reference, meaning that there should be no return value for `readData`. You may assume that exactly five data values will be entered.
- Define a second subprogram with the signature `int findMode(arr)`, which receives an array of floating point values (here called `arr`) and returns the mode of these values. The mode of a set of values is the most commonly occurring value in the set.
- Define a third subprogram with the signature `int findMean(arr)`, which receives an array of floating point values (here called `arr`) and returns the arithmetic mean of these values.
- Define a final subprogram with the signature `int findMedian(arr)`, which receives an array of floating point values (here called `arr`) and returns the median of the values in the array. The median of a set of values is the value in the middle position of a sorted version of set.
- Finally, your program should prompt the user for values that will be stored in the array. Your program should then determine the mode, mean, and median of the values stored in the array, and print out these values. Use the four subprograms you have written to perform these operations.

For example, assume the array contains the following values:

1.0	5.0	8.0	2.0	1.0

In the above example, the mode is 1.0 (because 1.0 occurs twice, and the other values occur only once each), the mean is 3.4, and the median is 2.0 (the middle value in the reordered list 1.0, 1.0, 2.0, 5.0, 8.0).

Note that in the above description the term "subprogram" refers to a unit of execution (you will probably know such units of execution as functions or methods). Some programming languages support different kinds of subprograms, and you should use the most appropriate type of subprogram for the task at hand.

It is also possible that language limitations may affect aspects of the subprograms in one way or another (for example, in terms of the return types allowed from subprograms). Should a subprogram not support a feature you would typically use, you must find another way to provide the required functionality. For example, if certain data types cannot be returned by a subprogram, consider whether you can pass a variable of the required type to the subprogram by reference. Similarly, if a value cannot be passed by reference, consider whether it can be returned instead. However, try to follow the above specification as closely as possible.

6 Marking

The implementation will count a total of 5 marks. Submit your implementations to the appropriate assignment upload slot. Do not upload any additional files other than your source code. Both the implementation and the correct execution of the program will be taken into account. Your program code will be assessed during the practical session in the week of **Monday, 11 May 2026**.

References

[1] PGI Compilers and Tools. Fortran reference guide: Version 2018, 2018.

[2] The `gfortran` team. Using GNU Fortran: For GCC version 6.3.0, 2016.